

WAD Homework — Technical Report

Field	Value
Assignment	Web Application Development — homework project
UI mode	Server-rendered HTML (MVC) — Jinja2 templates; not an SPA
Stack	Python, FastAPI, PostgreSQL (Alembic), Redis, JWT, GitHub OAuth
Author	<i>Yousef Abdah</i>
Repository	<i>WAD-Homework</i>

1. API structure

1.1 Overview

The application exposes:

- HTML routes — forms and redirects for browsers; authentication uses HttpOnly cookies (`access_token`, `refresh_token`).
- JSON API under prefix `/api` — REST-style resources; protected routes accept `Authorization: Bearer <access_token>` (preferred for API clients) or the same cookies as the browser.

Machine-readable contract: `GET /openapi.json`. Interactive explorer: `GET /docs` (Swagger UI).

1.2 Resource groups and routes

Health & diagnostics

Method	Path	Auth	Description
--------	------	------	-------------

GET	/health	No	Returns JSON flags for PostgreSQL and Redis reachability (200 if both OK, else 503).
-----	---------	----	--

MVC — pages & forms (browser)

Method	Path	Auth	Description
GET	/	No	Redirects to /chats if logged in, else /login.
GET	/login	No	Login form + link to GitHub OAuth.
POST	/login	No	Validates credentials; sets JWT + refresh cookies; redirects to /chats.
GET	/register	No	Registration form.
POST	/register	No	Creates user (bcrypt password); redirects to login.
POST	/logout	Yes (cookies)	Revokes refresh token in Redis; clears cookies; redirects to /login.
GET	/chats	Yes	Lists user's chats; form to create a chat.

POST	<code>/chats</code>	Yes	Creates a chat; redirects to <code>/chats/{id}</code> .
GET	<code>/chats/{chat_id}</code>	Yes	Chat thread + message history + send form.
POST	<code>/chats/{chat_id}/messages</code>	Yes	Appends user + assistant messages (LLM); redirects back to chat.
POST	<code>/chats/{chat_id}/messages/stream</code>	Yes	SSE (<code>text/event-stream</code>): streams assistant tokens, persists full reply (bonus).
GET	<code>/profile</code>	Yes	Shows username and linked OAuth accounts.

OAuth — GitHub

Method	Path	Auth	Description
GET	<code>/auth/github/login</code>	No	Stores OAuth <code>state</code> in Redis; redirects to GitHub authorize URL.

GET	/auth/github/callback	No	Validates <code>state</code> , exchanges <code>code</code> for tokens, upserts user + <code>oauth_accounts</code> , sets cookies; redirects to <code>/chats</code> .
-----	-----------------------	----	--

JSON API — authentication

Method	Path	Auth	Description
POST	/api/auth/token	No	OAuth2-style body: <code>username, password</code> (<code>application/x-www-form-urlencoded</code>). Returns <code>{ access_token, refresh_token, token_type }</code> .
POST	/api/auth/login	No	JSON <code>{ "username", "password" }</code> — same token response as above.
POST	/api/auth/register	No	JSON <code>{ "username", "password" }</code> — 201 + user JSON on success.
POST	/api/auth/refresh	Refresh	Optional query <code>refresh_token=...</code> , else refresh cookie. Returns new access + refresh; updates cookies.

POST	/api/auth/logout	Optional	Optional query <code>refresh_token=...</code> , else cookie; revokes refresh in Redis; clears cookies.
------	------------------	----------	--

JSON API — user & chats

Method	Path	Auth	Description
GET	/api/me	JWT	Current user (<code>id</code> , <code>username</code> , <code>is_admin</code>).
GET	/api/chats	JWT	Lists current user's chats.
POST	/api/chats	JWT	JSON body optional { <code>"title"</code> }; creates chat (201).
GET	/api/chats/{chat_id}	JWT	Single chat metadata (must belong to user).
GET	/api/chats/{chat_id}/messages	JWT	Message list; Redis cache-aside with TTL (invalidated on writes).
POST	/api/chats/{chat_id}/messages	JWT	JSON { <code>"content"</code> }; runs LLM; returns updated message list.
POST	/api/chats/{chat_id}/messages/stream	JWT	JSON { <code>"content"</code> }; SSE stream of tokens then persistence (bonus).

1.3 How clients use the API

Browser (MVC)

Use only HTML routes. The server sets HttpOnly cookies on login, GitHub callback, or successful `/api/auth/refresh`. Subsequent requests send cookies automatically. Refresh is applied when the access JWT expires but a valid refresh token exists (see `app/web/deps.py`).

Programmatic client (REST + JWT)

1. Obtain tokens: `POST /api/auth/token` (form) or `POST /api/auth/login` (JSON).
2. Call protected endpoints with header: `Authorization: Bearer <access_token>`.
3. When the access token expires: `POST /api/auth/refresh?refresh_token=<opaque_refresh>` (or rely on cookies if the client is cookie-aware).
4. Logout: `POST /api/auth/logout?refresh_token=...` to revoke the refresh entry in Redis.

GitHub sign-in

User hits `/auth/github/login` in the browser; after GitHub redirects back, the app issues the same cookie pair as password login.

1.4 OpenAPI excerpt (generated)

The canonical schema is generated at runtime. After starting the server:

- OpenAPI JSON: `http://127.0.0.1:8000/openapi.json`
- Swagger UI: `http://127.0.0.1:8000/docs`

For the PDF version of this report, you may paste a shortened JSON fragment from `/openapi.json` (paths `"/api/me"`, `"/api/chats"`, etc.) or export a screenshot of `/docs` showing the tagged api group.

1.5 Authentication & tokens (design summary)

- Passwords: bcrypt-hashed in `users.password_hash`.
- Access token: JWT (HS256), short TTL (env `ACCESS_TOKEN_TTL_SECONDS`, default 900 s).
- Refresh token: opaque random string stored in Redis under `refresh:<token>` with 30-day TTL (`REFRESH_TOKEN_TTL_DAYS`), rotated on refresh.
- GitHub: authorization code flow; see `docs/github-oauth-setup.md`.

2. Code structure

2.1 MVC layout (homework requirement)

Layer	Role	Location

Model	SQLAlchemy entities, DB schema	<code>app/models/</code>
View	Jinja2 HTML	<code>app/web/templates/</code>
Controller	HTTP routes, request/response wiring	<code>app/routers/pages.py</code> , <code>app/routers/oauth_github.py</code> , <code>app/main.py (/health)</code>
Service	Business logic (auth helpers, chats, LLM, GitHub HTTP, Redis cache/stream)	<code>app/services/</code>
Config / security	Settings, JWT, password hashing	<code>app/core/config.py</code> , <code>app/core/security.py</code> , <code>app/core/constants.py</code>
Persistence	Async SQLAlchemy session, Redis client	<code>app/db/database.py</code> , <code>app/db/redis_client.py</code>
Shared HTTP dependencies	Optional user / required user, DB session injection	<code>app/web/deps.py</code>
Cookies	Set/clear auth cookies	<code>app/web/cookies.py</code>
Migrations	Alembic revisions	<code>alembic/versions/</code>

Routers are registered in `app/main.py`: `pages` → `oauth_github` → `api` (prefix `/api` on the API router).

2.2 Important modules (short)

- `app/main.py` — FastAPI app factory, lifespan Redis connect/disconnect, `/health`.
- `app/routers/pages.py` — All server-rendered flows (auth forms, chats, profile, SSE stream for HTML).
- `app/routers/api.py` — JSON REST: auth, `me`, chats, messages, SSE for API clients.
- `app/routers/oauth_github.py` — GitHub authorize + callback.

- `app/services/auth_tokens.py` — Mint JWT + Redis refresh; exchange refresh rotation.
- `app/services/refresh_tokens.py` — Redis keys for refresh + OAuth `state`.
- `app/services/users.py` — Password users, GitHub user upsert.
- `app/services/chats.py` — List/create chats, load owned chat, append messages.
- `app/services/llm.py` — Non-stream and async token stream (llama-cpp or stub).
- `app/services/chat_cache.py`, `chat_stream.py` — Redis message cache (bonus), SSE pipeline (bonus).

2.3 How JWT, refresh, and Redis interact

1. On successful login (password, GitHub, or `/api/auth/login/token`), the server creates a JWT and an opaque refresh token.
2. The refresh token is stored in Redis with JSON payload `{"user_id": ...}` and TTL 30 days.
3. The browser stores both in HttpOnly cookies; API clients typically store tokens themselves and send Bearer access tokens.
4. When the access JWT is missing or invalid, the server may peek the refresh token in Redis, verify the user, revoke the old refresh, issue a new pair, and update cookies (`resolve_session_user / exchange_refresh_for_tokens`).
5. Logout deletes the refresh key in Redis and clears cookies.

3. UI (screenshots)

#	Screen	Suggested filename
1	Login	<code>project\docs\screenshots\Login.png</code>
2	Register	<code>docs\screenshots\Register.png</code>
3	Chats list	<code>docs\screenshots\Chats list.png</code>
4	Chat conversation	<code>project\docs\screenshots\Chat conversation.png</code>

5	Profile	<code>docs\screenshots\Profile.png</code>
---	---------	---

4. Database (schema and relationships)

4.1 Tables and columns

`users` — application accounts (password and/or OAuth-linked).

Column	Type	Notes
<code>id</code>	integer PK	Autoincrement
<code>username</code>	varchar(150), unique, nullable	Login name; unique
<code>password_hash</code>	varchar(255), nullable	bcrypt; null for GitHub-only users
<code>is_active</code>	boolean	Default true
<code>is_admin</code>	boolean	Default false
<code>created_at, updated_at</code>	timestampz	Server defaults / onupdate

`oauth_accounts` — linked external identities (GitHub).

Column	Type	Notes
<code>id</code>	integer PK	
<code>user_id</code>	FK → <code>users.id</code>	ON DELETE CASCADE

<code>provider</code>	<code>varchar(30)</code>	e.g. <code>github</code>
<code>provider_user_id</code>	<code>varchar(100)</code>	GitHub numeric id as string
<code>provider_login</code> , <code>provider_email</code>	nullable <code>varchar</code>	Profile cache
<code>created_at</code>	<code>timestampz</code>	
Constraint		<code>UNIQUE (provider, provider_user_id)</code>

`chats` — conversation threads per user.

Column	Type	Notes
<code>id</code>	integer PK	
<code>user_id</code>	FK → <code>users.id</code>	ON DELETE CASCADE
<code>title</code>	<code>varchar(200)</code> , nullable	Display title
<code>created_at</code> , <code>updated_at</code>	<code>timestampz</code>	

`messages` — turns inside a chat.

Column	Type	Notes
<code>id</code>	integer PK	
<code>chat_id</code>	FK → <code>chats.id</code>	ON DELETE CASCADE

<code>role</code>	<code>varchar(20)</code>	<code>user assistant system</code>
<code>content</code>	<code>text</code>	Message body
<code>created_at</code>	<code>timestampz</code>	

4.2 Relationships (textual)

- User 1 — N OAuthAccount — one user may have multiple providers (currently GitHub).
- User 1 — N Chat — chats belong to a single user.
- Chat 1 — N Message — ordered by `message.id`; deleting a chat deletes its messages (CASCADE).

4.3 ERD (diagram)

- Source (editable): `project/Report&Diagram/ERD.drawio`
- Submitted image: `project/Report&Diagram/ERD.jpg`

4.4 Migrations

Schema is applied with Alembic (`alembic upgrade head`). Initial revision:
`alembic/versions/0001_init_tables.py` (adjust filename if renamed in repo).

Instructions to run the app

- Copy `project/.env.example` → `.env`, set `DATABASE_URL`, `REDIS_URL`, `JWT_SECRET`, GitHub OAuth vars.
- `alembic upgrade head`
- `python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000`